**Department of Electrical Engineering**

**Indian Institute of Technology, Kharagpur**

**Algorithms, AI and ML Laboratory (EE22202)**

**Spring, 2025-26**

# Report 10: Semantic Image Segmentation using UNet

**Name: Arijit Dey**

**Roll No: 24IE10001**

# Semantic Image Segmentation using UNet

## Dataset and preprocessing

We define a helper class for loading and tranforming the Oxford-IIIT Pet dataset. It is responsible for providing easy access to various images of the dataset using a simple array access like notation. Specificially we perform these steps:

1. Load the RGB image and the corresponding trimap mask using matched filenames.
2. Resize both to a fixed target size to standardize the input resolution.
3. Applie normalization and convert the mask to zero-indexed class labels.

**Dataset class and transforms**

```python
class PetSegmentationDataset(Dataset):
    # ... OTHER FUNCTIONS OMITTED

    def __getitem__(self, idx):
        img_name = self.images[idx]
        img_path = os.path.join(self.image_dir, img_name)
        # Assuming masks have the same filename but .png extension
        mask_path = os.path.join(self.mask_dir, img_name.split(".")[0] + ".png")
        image = Image.open(img_path).convert("RGB")
        mask = Image.open(mask_path)  # 1 - channel trimap (1 , 2 , 3)

        resize_transform = transforms.Resize(self.target_size)
        image = resize_transform(image)
        mask = resize_transform(mask)

        if self.transform:
            image = self.transform(image)
            # Ensure mask is converted to tensor and classes are 0 - indexed (0 , 1 , 2)
            mask = torch.as_tensor(np.array(mask), dtype=torch.long) - 1
            return image, mask

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]
)
```

## Data access and dataset setup

We setup the paths where the images and their annotations are stored and intialize the `PetSegmentationDataset` class. We make a 70/15/15 split for training, validation and testing dataset. For training the network we use a batch size of 16. The device is set to `cuda` if a CUDA-enabled GPU is detected, otherwise it is set to `cpu`[1].

**Dataset split and sample visualization**

```python
data_dir = "./drive/MyDrive/AI ML Expt 10/"
```

[1]This experiment was performed on a T4 GPU

```python
image_dir = os.path.join(data_dir, "images")
mask_dir = os.path.join(data_dir, "annotations/trimaps")

dataset = PetSegmentationDataset(image_dir, mask_dir, transform=transform)

torch.backends.cudnn.benchmark = True

train_size = int(0.7 * len(dataset))
val_size = int(0.15 * len(dataset))
test_size = len(dataset) - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(dataset, [train_size, val_size, test_size])

BATCH_SIZE = 16
NUM_WORKERS = 4

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
Result
Dataset loaded with 7390 items with split: Train 5173, Val 1108, Test 1109
```

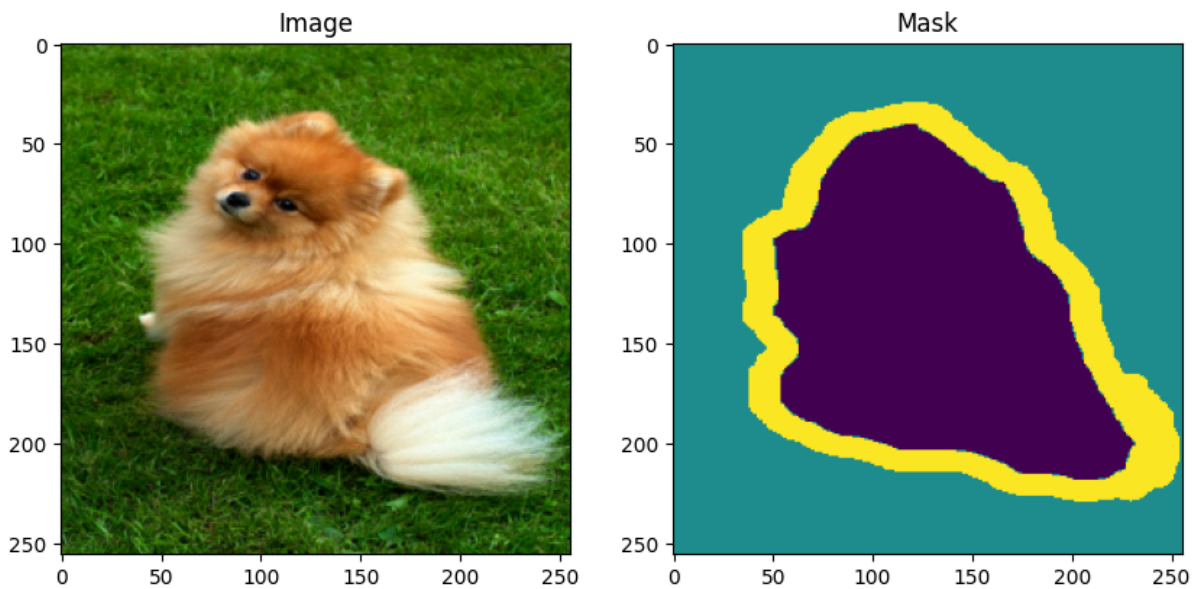


Figure 1: Sample image and trimap mask.

## UNet architecture

This section defines the UNet encoder-decoder architecture with optional skip connections, batch normalization, and dropout.

### The `forward()` function in `DoubleConv`

1. Applies a two-layer convolutional block with optional batch normalization and dropout.

### The `forward()` function in `UNet`

1. Encodes inputs with successive downsampling blocks and max pooling.
2. Processes the bottleneck feature map before upsampling in the decoder.
3. Concatenates encoder features via skip connections when enabled.
4. Produces per-pixel class logits through the final 1x1 convolution.

**UNet building blocks**

```python
class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels, use_bn=False, drop_p=0.0):
        super(DoubleConv, self).__init__()
        layers = [
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels) if use_bn else nn.Identity(),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels) if use_bn else nn.Identity(),
            nn.ReLU(inplace=True),
        ]
        if drop_p > 0:
            layers.append(nn.Dropout(drop_p))
        self.conv = nn.Sequential(*layers)

    def forward(self, x):
        return self.conv(x)


class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=3, use_skip=True, use_bn=False, drop_p=0.0):
        super(UNet, self).__init__()
        self.use_skip = use_skip

        self.enc1 = DoubleConv(in_channels, 64, use_bn=use_bn)
        self.enc2 = DoubleConv(64, 128, use_bn=use_bn)
        self.enc3 = DoubleConv(128, 256, use_bn=use_bn)
        self.enc4 = DoubleConv(256, 512, use_bn=use_bn)

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.bottleneck = DoubleConv(512, 1024, use_bn=use_bn, drop_p=drop_p)

        self.up4 = nn.ConvTranspose2d(1024, 512, kernel_size=2, stride=2)
        self.dec4 = DoubleConv(1024 if use_skip else 512, 512, use_bn=use_bn)
        self.up3 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
        self.dec3 = DoubleConv(512 if use_skip else 256, 256, use_bn=use_bn)
        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.dec2 = DoubleConv(256 if use_skip else 128, 128, use_bn=use_bn)
        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
```

```python
        self.dec1 = DoubleConv(128 if use_skip else 64, 64, use_bn=use_bn)

        self.out_conv = nn.Conv2d(64, num_classes, kernel_size=1)

    def forward(self, x):
        e1 = self.enc1(x)
        p1 = self.pool(e1)
        e2 = self.enc2(p1)
        p2 = self.pool(e2)
        e3 = self.enc3(p2)
        p3 = self.pool(e3)
        e4 = self.enc4(p3)
        p4 = self.pool(e4)

        b = self.bottleneck(p4)

        d4 = self.up4(b)
        if self.use_skip:
            d4 = torch.cat((e4, d4), dim=1)
        d4 = self.dec4(d4)

        d3 = self.up3(d4)
        if self.use_skip:
            d3 = torch.cat((e3, d3), dim=1)
        d3 = self.dec3(d3)

        d2 = self.up2(d3)
        if self.use_skip:
            d2 = torch.cat((e2, d2), dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        if self.use_skip:
            d1 = torch.cat((e1, d1), dim=1)
        d1 = self.dec1(d1)

        return self.out_conv(d1)
```

## Loss functions, metrics, and training utilities

Loss functions and evaluation metrics are defined, followed by training and evaluation helpers that track loss curves and compute mIoU and Dice scores.

### The `forward()` function in `DiceLoss`

1. Computes the softmax probabilities and compares them against one-hot encoded targets.
2. Aggregates per-class Dice scores and returns the mean Dice loss.

**Losses, metrics, and training helpers**

```python
class DiceLoss(nn.Module):
    def __init__(self, smooth=1.0):
        super(DiceLoss, self).__init__()
```

```python
        self.smooth = smooth

    def forward(self, logits, targets):
        num_classes = logits.shape[1]
        probs = F.softmax(logits, dim=1)

        targets_one_hot = F.one_hot(targets, num_classes).permute(0, 3, 1, 2).float()

        dims = (0, 2, 3)
        intersection = torch.sum(probs * targets_one_hot, dims)
        cardinality = torch.sum(probs + targets_one_hot, dims)

        dice_score = (2. * intersection + self.smooth) / (cardinality + self.smooth)
        return 1 - torch.mean(dice_score)


def calculate_metrics(outputs, targets, num_classes=3):
    preds = torch.argmax(outputs, dim=1)

    ious = torch.zeros(num_classes, device=outputs.device)
    dice_coeffs = torch.zeros(num_classes, device=outputs.device)

    for cls in range(num_classes):
        pred_cls = (preds == cls)
        target_cls = (targets == cls)

        intersection = (pred_cls & target_cls).float().sum()
        union = (pred_cls | target_cls).float().sum()

        iou = torch.where(union == 0, torch.tensor(1.0, device=outputs.device), intersection / (union + 1e-7))
        dice = (2 * intersection) / (pred_cls.float().sum() + target_cls.float().sum() + 1e-7)

        ious[cls] = iou
        dice_coeffs[cls] = dice

    return ious.mean().item(), dice_coeffs.mean().item()


def train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs=30, device='cpu'):
    train_losses = []
    val_losses = []

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        for images, masks in tqdm(train_loader):
            images, masks = images.to(device, non_blocking=True), masks.to(device, non_blocking=True)
```

```python

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, masks)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        avg_train_loss = running_loss / len(train_loader)
        train_losses.append(avg_train_loss)

        model.eval()
        running_val_loss = 0.0
        with torch.no_grad():
            for images, masks in val_loader:
                images, masks = images.to(device, non_blocking=True), masks.to(device, non_blocking=True)
                outputs = model(images)
                loss = criterion(outputs, masks)
                running_val_loss += loss.item()

        avg_val_loss = running_val_loss / len(val_loader)
        val_losses.append(avg_val_loss)

    return train_losses, val_losses


def evaluate_model(model, test_loader, device='cpu'):
    model.eval()
    all_mious = []
    all_dice = []
    with torch.no_grad():
        for images, masks in test_loader:
            images, masks = images.to(device, non_blocking=True), masks.to(device, non_blocking=True)
            outputs = model(images)
            miou, dice = calculate_metrics(outputs, masks)
            all_mious.append(miou)
            all_dice.append(dice)

    return np.mean(all_mious), np.mean(all_dice)
```

## Experiment 1: Baseline UNet

The baseline experiment trains a standard UNet with skip connections using cross-entropy loss.

**Baseline UNet training**

```python
model_1 = UNet(use_skip=True, use_bn=False, drop_p=0.0).to(device)
trainable_params_1 = sum(p.numel() for p in model_1.parameters() if p.requires_grad)

```

```
criterion_1 = nn.CrossEntropyLoss()
optimizer_1 = optim.Adam(model_1.parameters(), lr=0.001)

train_losses_1, val_losses_1 = train_model(model_1, train_loader, val_loader, criterion_1,
optimizer_1, num_epochs=30, device=device)

plot_curves(train_losses_1, val_losses_1, "Baseline UNet: Loss Curves")
miou_1, dice_1 = evaluate_model(model_1, test_loader, device=device)
```

**Result**

```
Total Trainable Parameters: 31031875
Test mIoU: 0.7270, Test Dice Coefficient: 0.8302
```

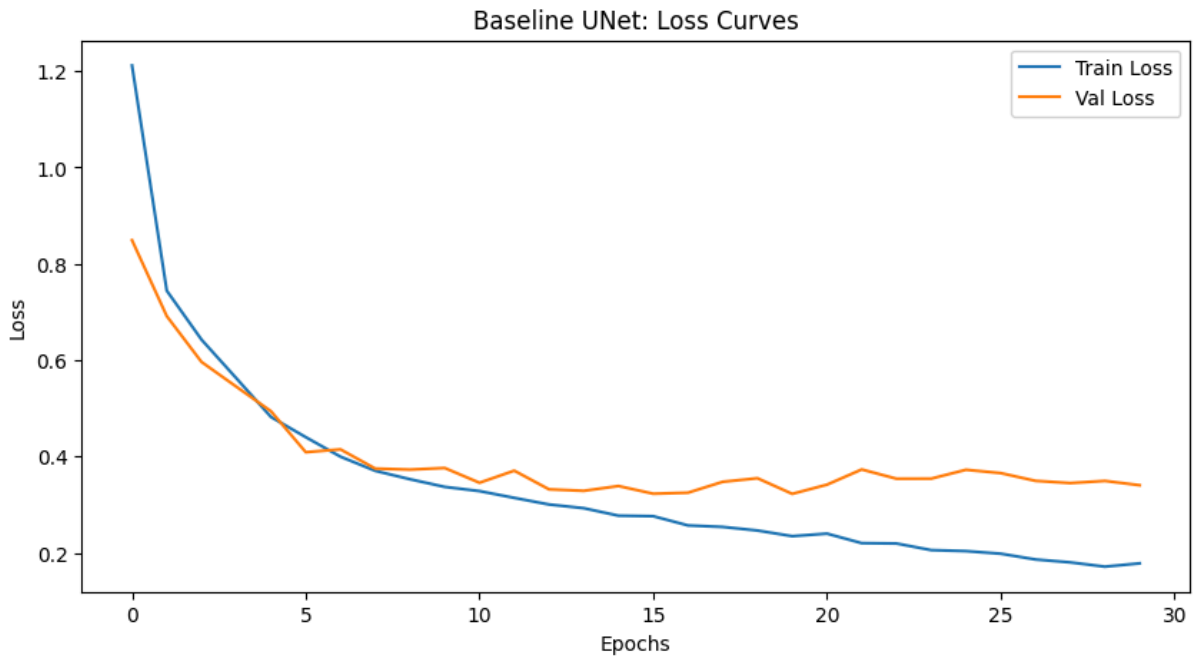


Figure 2: Baseline UNet loss curves.

## Observations

The baseline model reports 31,031,875 trainable parameters and achieves a test mIoU of 0.7270 with a Dice coefficient of 0.8302, while the plotted curves provide a visual comparison of training and validation loss over epochs.

## Experiment 2: Architectural ablation (no skip connections)

This experiment disables skip connections to assess their impact on segmentation performance.

**UNet without skip connections**

```
model_2 = UNet(use_skip=False, use_bn=False, drop_p=0.0).to(device)
trainable_params_2 = sum(p.numel() for p in model_2.parameters() if p.requires_grad)

criterion_2 = nn.CrossEntropyLoss()
optimizer_2 = optim.Adam(model_2.parameters(), lr=0.001)

train_losses_2, val_losses_2 = train_model(model_2, train_loader, val_loader, criterion_2,
optimizer_2, num_epochs=30, device=device)
```

```python

plot_curves(train_losses_2, val_losses_2, "No Skip Connections: Loss Curves")
miou_2, dice_2 = evaluate_model(model_2, test_loader, device=device)
```

```
Result
Total Trainable Parameters: 27898435
Test mIoU: 0.5376, Test Dice Coefficient: 0.6236
```

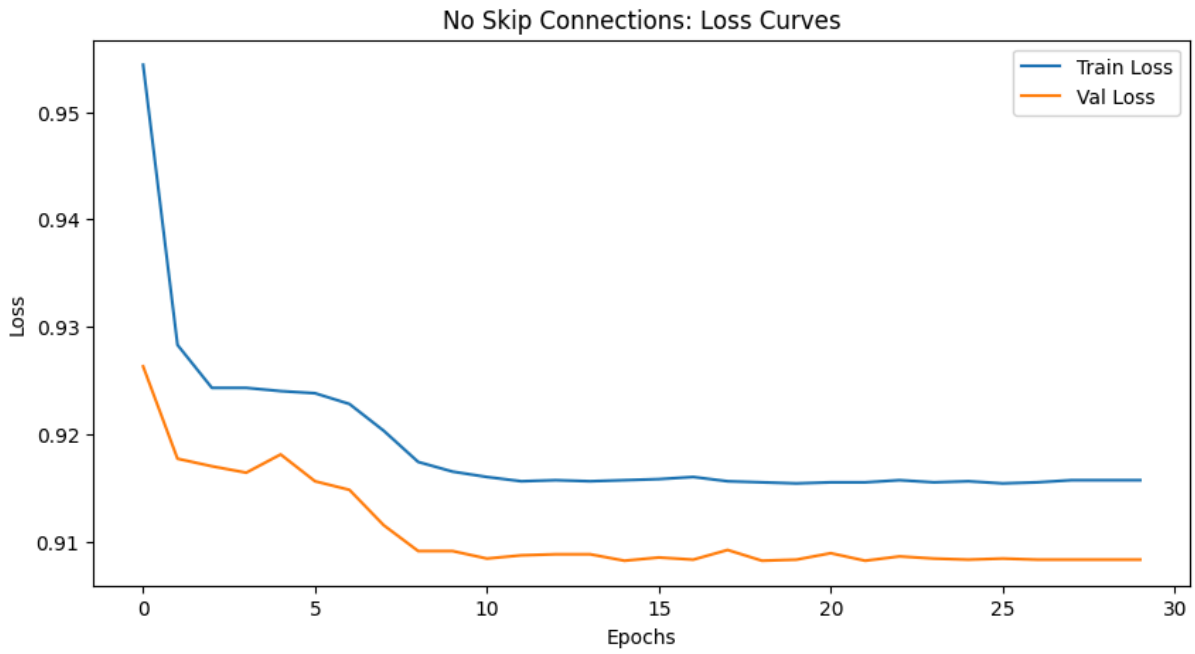


Figure 3: UNet without skip connections loss curves.

## Observations

The skip-connection ablation reduces the parameter count to 27,898,435 and yields lower test performance with mIoU 0.5376 and Dice 0.6236, alongside the displayed training and validation loss curves.

## Experiment 3: Loss function ablation (cross-entropy + Dice)

This experiment introduces a combined cross-entropy and Dice loss to evaluate its effect on performance.

### The `forward()` function in `CEDiceLoss`

1. Computes a weighted sum of cross-entropy loss and Dice loss for multi-class segmentation.

```
Combined CE and Dice loss
class CEDiceLoss(nn.Module):
    def __init__(self,
                 ce_weight=1.0,
                 dice_weight=1.0):
        # -- SETUP CODE --

    def forward(self, logits, targets):
        return self.ce_weight * self.ce(logits, targets) + self.dice_weight *
        self.dice(logits, targets)
```

```
UNet with CE + Dice loss
model_3 = UNet(use_skip=True, use_bn=False, drop_p=0.0).to(device)
trainable_params_3 = sum(p.numel() for p in model_3.parameters() if p.requires_grad)

criterion_3 = CEDiceLoss()
optimizer_3 = optim.Adam(model_3.parameters(), lr=0.001)

train_losses_3, val_losses_3 = train_model(model_3, train_loader, val_loader, criterion_3,
optimizer_3, num_epochs=30, device=device)

plot_curves(train_losses_3, val_losses_3, "CE + Dice Loss: Loss Curves")
miou_3, dice_3 = evaluate_model(model_3, test_loader, device=device)
```

```
Result
Total Trainable Parameters: 31031875
Test mIoU: 0.5873, Test Dice Coefficient: 0.7251
```

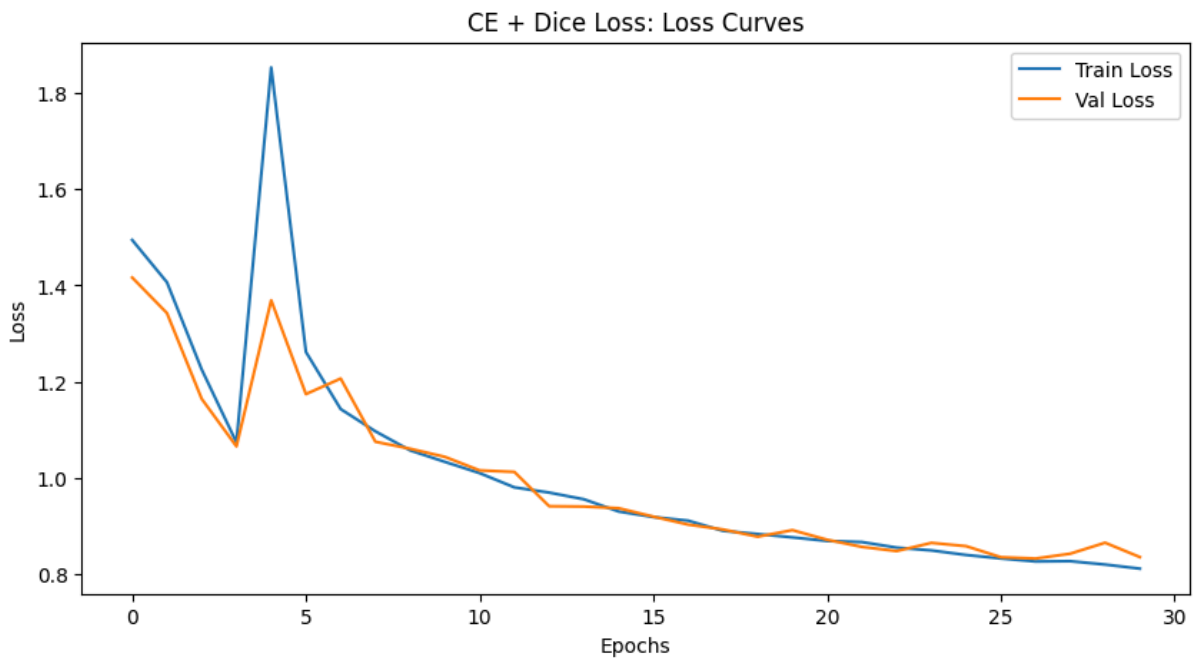


Figure 4: CE + Dice loss curves.

## Observations

With the combined CE + Dice loss, the model retains 31,031,875 parameters and reports test mIoU 0.5873 with Dice 0.7251, and the loss curves visualize the training and validation trends.

## Experiment 4: Regularization (batch normalization and dropout)

This experiment applies batch normalization and dropout in the UNet to study regularization effects.

```
UNet with batch normalization and dropout
model_4 = UNet(use_skip=True, use_bn=True, drop_p=0.3).to(device)
trainable_params_4 = sum(p.numel() for p in model_4.parameters() if p.requires_grad)

criterion_4 = CEDiceLoss()
```

```python
optimizer_4 = optim.Adam(model_4.parameters(), lr=0.001)

train_losses_4, val_losses_4 = train_model(model_4, train_loader, val_loader, criterion_4, optimizer_4, num_epochs=30, device=device)

plot_curves(train_losses_4, val_losses_4, "BN + Dropout (CE + Dice): Loss Curves")
miou_4, dice_4 = evaluate_model(model_4, test_loader, device=device)
```

```
Result
Total Trainable Parameters: 31043651
Test mIoU: 0.4638, Test Dice Coefficient: 0.7618
```

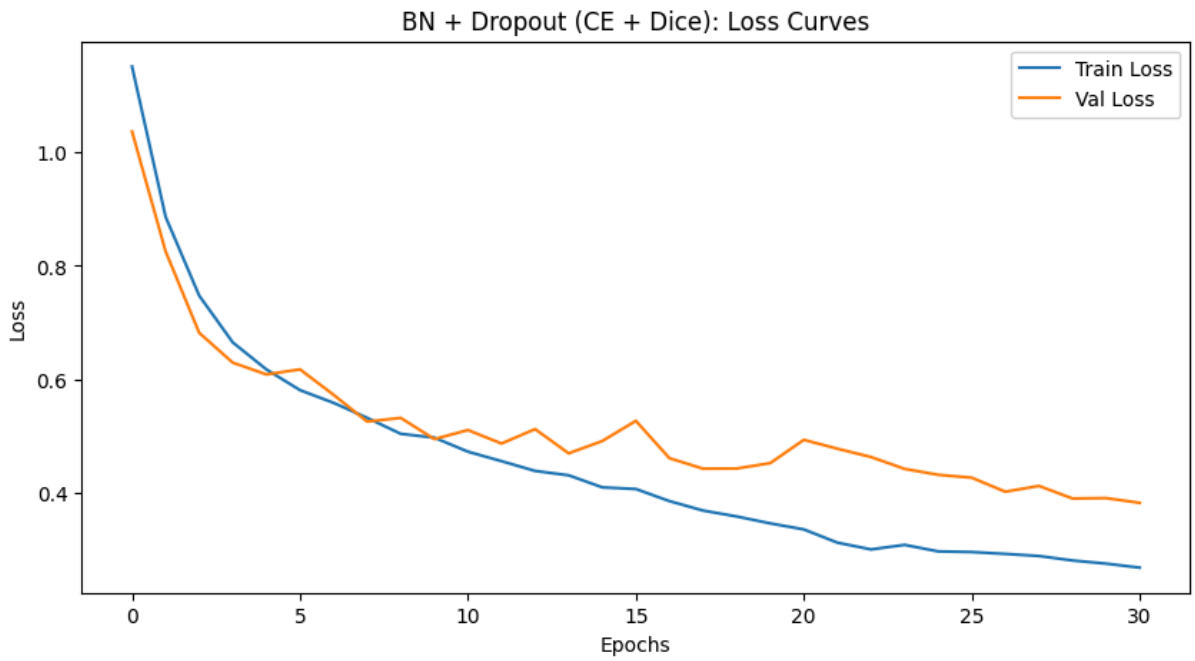


Figure 5: Batch normalization and dropout loss curves.

## Observations

The regularized UNet has 31,043,651 trainable parameters and achieves test mIoU 0.4638 with Dice 0.7618, while the plotted curves summarize the loss trajectory under batch normalization and dropout.

## Conclusion

1. A custom dataset pipeline standardizes image-mask pairs to 256x256 resolution and confirms dataset integrity through a visual sample.
2. The baseline UNet with skip connections attains the best test performance among the experiments, with mIoU 0.7270 and Dice 0.8302.
3. Removing skip connections substantially reduces segmentation quality, indicating their importance for preserving spatial detail.
4. The combined CE + Dice loss improves Dice compared to the no-skip setup but underperforms the baseline in mIoU.
5. Batch normalization with dropout yields lower mIoU than the baseline while maintaining a relatively higher Dice coefficient.